Laboratory Activity: Tracing Selection Sort in Java

Name: Rodriguez, Miles J. **Date:** Oct 28, 2026

Class Section: BSIT 2H-G1

I. Objectives

By the end of this activity, students will be able to:

- **Understand** the core mechanics of the Selection Sort algorithm in Java, specifically how the sorted and unsorted regions interact.
- **Trace** the state of an array and the behavior of variables (minIndex, temp, i, j) through multiple iterations using a unique dataset.
- **Analyze** the step-by-step swapping mechanism using a temporary holding variable to prevent data loss.

II. Instructions

1. **Analyze the Code:** Review the provided Java program for Selection Sort below.
2. **Execute the Trace:** Simulate how the algorithm processes the **new** initial array: int[] arr = {42, 11, 28, 7, 34};.
3. **Fill out the Tables:** Complete all the blank cells in the Iteration Trace tables, Swap Step breakdown tables, and final summary table for each pass.
4. **Submission:** Submit your completed answer sheet either digitally or in print as instructed by your professor.

III. Sample Program for Analysis

Study the complete, working Java program below, which implements Selection Sort in ascending order:

```
public class SelectionSortAscending {
  public static void main(String[] args) {
    int[] arr = {42, 11, 28, 7, 34};
    selectionSort(arr);
```

```
        System.out.print("Sorted Array: ");
        for (int num : arr) {
            System.out.print(num + " ");
        }
    }

    static void selectionSort(int[] arr) {
        // Outer loop controls the boundary of the sorted region
        for (int i = 0; i < arr.length - 1; i++) {
            int minIndex = i; // Assume the first element of unsorted region is the minimum

            // Inner loop scans the remaining unsorted elements
            for (int j = i + 1; j < arr.length; j++) {
                if (arr[j] < arr[minIndex]) {
                    minIndex = j; // Update minIndex if a smaller element is found
                }
            }

            // Swap step: Move the found minimum element to the front of the unsorted region
            int temp = arr[minIndex];
            arr[minIndex] = arr[i];
            arr[i] = temp;
        }
    }
}
```

IV. Activity Exercises

Given the initial array arr = {42, 11, 28, 7, 34} (with arr.length = 5), fill out the tracking and swap tables below for each iteration.

Part 1: Iteration 1 (i = 0)

Start State: minIndex = 0

A. Inner Loop Trace Table

j	Condition (j<arr.length)	Comparison (arr[j]<arr[minIndex])	Result (True/False)	Action / State Update	Current Array State
1	1 < 5 → TRUE	11 < 42	TRUE	minIndex = 1	{42, 11, 28, 7, 34}
2	2 < 5 → TRUE	28 < 11	FALSE		{42, 11, 28, 7, 34}
3	3 < 5 → TRUE	7 < 11	TRUE	minIndex = 3	{42, 11, 28, 7, 34}
4	4 < 5 → TRUE	34 < 7	FALSE	-	{42, 11, 28, 7, 34}
5	5 < 5 → FALSE	–	–	-	-

B. Swap Step Breakdown Table

Line of Code	What Happens (The Action)	Value of temp	Value at arr[i] (arr[0])	Value at arr[minIndex]	Array State After Step
Initial State	Before the swap begins	–	42	7	{42, 11, 28, 7, 34}
int temp = arr[minIndex];	Saves the minimum value (7) to temp	7	42	7	{42, 11, 28, 7, 34}
arr[minIndex] = arr[i];	Moves 42 on 7's original place	7	42	42	{42, 11, 28, 42, 34}
arr[i] = temp;	Put 7 in place of 42	7	7	42	{7, 11, 28, 42, 34}

Part 2: Iteration 2 (i = 1)

Start State: minIndex = 1

A. Inner Loop Trace Table //7, 11, 28, 42, 34

j	Condition (j<arr.length)	Comparison (arr[j]<arr[minIndex])	Result (True/False)	Action / State Update	Current Array State
2	2 < 5 → TRUE	28 < 11	FALSE	-	{7, 11, 28, 42, 34}
3	3 < 5 → TRUE	42 < 11	FALSE		{7, 11, 28, 42, 34}
4	4 < 5 → TRUE	34 < 11	FALSE		{7, 11, 28, 42, 34}
5	5 < 5	–	–		

B. Swap Step Breakdown Table

Line of Code	What Happens (The Action)	Value of temp	Value at arr[i] (arr[1])	Value at arr[minIndex]	Array State After Step
Initial State	Before the swap begins	–	11	11	{7, 11, 28, 42, 34}
int temp = arr[minIndex];	Store minIndex (11) to temp	11	11	11	{7, 11, 28, 42, 34}
arr[minIndex] = arr[i];	No changes	11	11	11	{7, 11, 28, 42, 34}
arr[i] = temp;	Put temp (11) on the position of I (11)	11	11	11	{7, 11, 28, 42, 34}

Part 3: Iteration 3 (i = 2)

Start State: minIndex = 2

A. Inner Loop Trace Table //7, 11, 28, 42, 34

j	Condition (j<arr.length)	Comparison (arr[j]<arr[minIndex])	Result (True/False)	Action / State Update	Current Array State
3	3 < 5	42 < 28	FALSE		{7, 11, 28, 42, 34}
4	4 < 5	34 < 28	FALSE		{7, 11, 28, 42, 34}
5	5 < 5	–	–		{7, 11, 28, 42, 34}

B. Swap Step Breakdown Table

Line of Code	What Happens (The Action)	Value of temp	Value at arr[i] (arr[2])	Value at arr[minIndex]	Array State After Step
Initial State	Before the swap begins	–	28	28	{7, 11, 28, 42, 34}
int temp = arr[minIndex];	Store minIndex (28) to temp	28	28	28	{7, 11, 28, 42, 34}
arr[minIndex] = arr[i];	No changes	28	28	28	{7, 11, 28, 42, 34}
arr[i] = temp;	Put temp (28) on the position of I (28)	28	28	28	{7, 11, 28, 42, 34}

Part 4: Iteration 4 (I = 3)

Start State: minIndex = 3

A. Inner Loop Trace Table {7, 11, 28, 42, 34}

j	Condition (j<arr.length)	Comparison (arr[j]<arr[minIndex])	Result (True/False)	Action / State Update	Current Array State
4	4 < 5	34 < 42	TRUE	minIndex = 4	{7, 11, 28, 42, 34}
5	5 < 5	–	–		{7, 11, 28, 42, 34}

B. Swap Step Breakdown Table

Line of Code	What Happens (The Action)	Value of temp	Value at arr[i] (arr[3])	Value at arr[minIndex]	Array State After Step
Initial State	Before the swap begins	—	42	34	{7, 11, 28, 42, 34}
int temp = arr[minIndex];		34	42	34	{7, 11, 28, 42, 34}
arr[minIndex] = arr[i];		34	42	42	{7, 11, 28, 42, 42}
arr[i] = temp;		34	34	42	{7, 11, 28, 34, 42}

Part 5: Step-by-Step Iteration Summary Table

Fill out this overall summary of the passes once you have completed the individual iterations.

Iteration	Target Index (i)	Elements Scanned in Unsorted Region	Minimum Found (minIndex)	Array State After Swap
Iteration 1	0	{11, 28, 7, 34 }	7 (at index 3)	{7, 11, 28, 42, 34}
Iteration 2	1	{28, 42 , 34}	11 (at index 1) //No changes	{7, 11, 28, 42, 34}
Iteration 3	2	{42, 34}	28 (at index 2) //No changes	{7, 11, 28, 42, 34}
Iteration 4	3	{34}	34 (at index 4)	{7, 11, 28, 34, 42}
Final Sorted Output:				{7, 11, 28, 34, 42}

V. Grading Rubric

Criteria	Excellent (4 Points)	Proficient (3 Points)	Developing (2 Points)	Beginning (1-0 Points)
Accuracy of Tracing (Iteration Tables)	All loop conditions, comparisons, and boolean results are correctly identified and calculated across all iterations.	Most comparison results and index trackers are accurate with minor errors.	Several calculation or logic errors found in tracking the inner loop.	Tracing tables are largely incomplete or incorrect.
Understanding of Swap Mechanism	Clearly and accurately explains variable states (temp, arr[i], arr[minIndex]) line-by-line without data loss.	Explains the swap correctly but misses minor step-by-step state details.	Demonstrates partial understanding of the temporary holding variable concept.	Fails to trace the variable values through the swap instructions correctly.
Completeness & Neatness	All blanks are filled out properly with logical consistency and clear formatting/handwriting.	Almost all fields are complete with minimal blanks.	Several major sections or fields left blank.	Incomplete submission with little to no effort shown.